

DEVELOPPEMENT D'APPLICATIONS WEB

JavaScript

Contenu de la partie JavaScript

1. Introduction à JavaScript
2. Intégration de JavaScript dans l'HTML
3. Syntaxe du langage
4. La POO avec JavaScript
5. Syntaxe et manipulation JSON
6. Manipulation DOM
7. Gestion des événements utilisateur

Introduction à JavaScript

- JavaScript (JS) est un langage de scripts qui peut être incorporé aux balises HTML. JS est exécuté par le navigateur ;
- Son rôle est d'améliorer la présentation et l'**interactivité** des pages Web ;
- JavaScript offre des « gestionnaires d'événement » qui reconnaissent et réagissent aux demandes du client (comme les mouvements de la souris, clics sur les touches du clavier, etc.)



Intégration de JavaScript dans HTML

JavaScript peut être intégré n'importe où dans le document HTML. Il est aussi possible de l'utiliser plusieurs fois dans le même document HTML.

Méthode 1 : Code JavaScript intégré au document html

```
<script>  
/* ou // code js*/  
</script>
```

Méthode 2 : Code JavaScript externe

```
<script language="javascript" src="monScript.js"> </script>
```

Méthode 3 : Pseudo-URL

```
<a href="JavaScript:window.alert('Welcome to JavaScript!');">clickez ici</a>
```

Identifiants JavaScript

- Un identifiant en JS correspond au nom d'une variable. Ce nom doit être unique.
- Ces identifiants peuvent être des noms courts (comme x et y) ou bien des noms plus descriptifs (comme note, total, NomComplet, etc.)
- Un identifiant peut contenir des lettres, des chiffres, des traits de soulignement (_) et des signes dollar (\$).
- Un identifiant peut commencer par une lettre, un signe \$ ou bien un signe _
- JS est sensible à la casse (y et Y sont considérés comme des variables différentes).
- Un identifiant ne peut pas être un mot réservé du langage.

Types de données JavaScript

JavaScript est un langage faiblement typé : le type d'une variable est défini au moment de l'exécution.

On peut déclarer une variable JS en utilisant les mots-clés suivants :

- **var** : utilisé pour déclarer une variable globale (function scope) ;
- **let** : utilisé pour déclarer une variable dont la portée est limitée à un bloc (block scope) ;
- **const** : permet de déclarer une variable qui doit avoir une valeur initiale et ne peut pas être réassignée

Exemples:

- Déclaration explicite (en utilisant le mot clé var) :

```
var nom_variable = new Type de la variable;
```

```
var nom_variable;
```

- Déclaration implicite (sans utiliser var, on écrit le nom de la variable suivie du caractère d'affectation et de la valeur à affecter)

```
Numéro = 1 ; Prénom = "xyz" ;
```

Types de données JavaScript (exemples)

Déclaration des variable booléennes :

```
var test=new Boolean(true) ;
```

```
test=new Boolean(1) ;
```

```
let test = true.
```

Déclaration des chaînes de caractères :

```
var chaine = "Bonjour";
```

Déclaration des nombres :

```
var entier = 60; //un entier ;
```

```
let pi = 3.14; //un nombre réel.
```

Déclaration de plusieurs variables :

```
var variable3 = 2, variable4 = "mon texte d'initialisation";
```

Déclaration sans initialisation :

Une variable déclarée sans valeur aura la valeur undefined.

Portée des variables (variable scope)

Portée du bloc : (Block scope)

- En utilisant le mot clé **let**, les variables déclarées à l'intérieur d'un bloc { } ne sont pas accessibles depuis l'extérieur du bloc :

```
{      let x = 2;      }  
// x n'est pas accessible ici
```

- En utilisant le mot clé **var**, les variables déclarées à l'intérieur d'un bloc { } sont accessibles depuis l'extérieur du bloc.

```
{      var x = 2;      }  
// x est accessible ici
```

Portée locale : (Function scope)

- Les variables déclarées dans une fonction JavaScript deviennent LOCALES à la fonction : Ils ne sont accessibles qu'à partir de la fonction.

```
function Test()  
{  
    var x = "test1";  
    let y = "test2";  
    const z = "test3";  
}  
//x, y et z ne sont pas  
accessibles en dehors de la  
fonction
```

Portée globale : (Global scope)

- Une variable déclarée en dehors d'une fonction, devient GLOBAL. Les variables globales sont accessibles de n'importe où dans un programme JavaScript..

Opérateurs arithmétiques JavaScript

Opérateur	Signification
+	Addition
-	Soustraction
*	Multiplication
**	Puissance
/	Division
%	Modulo (Reste de la division)
++	Incrémentation
--	Décrémentation

Opérateurs d'affectation JavaScript

Opérateur	Exemple d'utilisation	Signification	Exemple complet	Résultat
=	<code>x = y</code>	x prend la valeur de y	<code>let x = 5</code>	x vaut 5
+=	<code>x += y</code>	$x = x + y$	<code>let x = 10; x += 5;</code>	x vaut 15
-=	<code>x -= y</code>	$x = x - y$	<code>let x = 10; x -= 5;</code>	x vaut 5
*=	<code>x *= y</code>	$x = x * y$	<code>let x = 10; x *= 5;</code>	x vaut 50
/=	<code>x /= y</code>	$x = x / y$	<code>let x = 10; x /= 5;</code>	x vaut 2
%=	<code>x %= y</code>	$x = x \% y$	<code>let x = 10; x %= 5;</code>	x vaut 0
**=	<code>x **= y</code>	$x = x ** y \Leftrightarrow$ x = x à la puissance y	<code>let x = 3; x **= 2;</code>	x vaut 9

Opérateurs de comparaison en JavaScript

Opérateur	Signification	Exemple
<code>==</code>	Egal à (comparaison des valeurs)	<code>let x = 5; let y = "5"; let z=(x==y); //z=true</code>
<code>===</code>	Egal à (comparaison de la valeur et du type)	<code>let x = 5; let y = "5"; let z=(x===y); //z=false</code>
<code>!=</code>	Différent de (n'est pas égal à)	<code>let x = 5; let y = 5; let z=(x!=y); //z=false</code>
<code>!==</code>	Type ou valeur différente	<code>let x = 5; let y = "5"; let z=(x!==y); //z=true</code>
<code>></code>	Supérieur à	<code>let x = 5; let y = 5; let z=(x>y); //z=false</code>
<code><</code>	Inférieur à	<code>let x = 5; let y = 5; let z=(x<y); //z=false</code>
<code>>=</code>	Supérieur ou égal à	<code>let x = 5; let y = 5; let z=(x>=y); //z=true</code>
<code><=</code>	Inférieur ou égal à	<code>let x = 5; let y = 5; let z=(x<=y); //z=true</code>

Opérateurs logiques en JavaScript

Opérateur	Signification
&&	ET logique
 	OU logique
!	NON logique

Opérateurs de type en JavaScript

Opérateur	Signification	Exemple
typeof	Retourne le type de la variable	typeof(5) retourne "number" typeof("5") retourne "string"
instanceof	Retourne true si l'objet est une instance de la classe donnée en paramètre	console.log("JavaScript" instanceof String);

Possibilités d'affichage avec JavaScript

document.write()

```
<script language="JavaScript">  
    document.write("Hello world!");  
</script>
```

alert()

```
<script language="JavaScript">  
    alert("Hello world!");  
</script>
```

console.log()

```
<script language="JavaScript">  
    console.log("Hello world!");  
</script>
```

Les entrées en Javascript

Prompt : affiche une boîte de dialogue avec une zone de saisie

- La méthode **prompt** (boite d'invite) propose un champ comportant une entrée à compléter par l'utilisateur avec une valeur par défaut.
- En cliquant sur OK, la méthode renvoie la valeur tapée par l'utilisateur ou la réponse proposée par défaut. Si l'utilisateur clique sur Annuler (Cancel), la valeur **null** est alors renvoyée.

```
var prenom = prompt("Quel est votre prénom?");  
document.write(prenom);
```

Les formulaires : les contrôles des formulaires comme la zone <input>

- Javascript peut récupérer les données de n'importe quel élément html par la méthode : **document.getElementById(id)** qui prend en paramètre l'identifiant de l'objet.

```
<!DOCTYPE html>  
<html>  
<body>  
<input type="text" id="prenom">  
<button  
onClick="alert(document.getElementById('prenom')  
.value)">Afficher</button>  
</body>  
</html>
```

Structures alternatives

```
if (condition)
{
    // bloc d'instructions à exécuter si la condition est vraie
}
```

```
if (condition)
{
    // bloc d'instructions à exécuter si la condition est vraie
}
else
{
    // bloc d'instructions à exécuter si la condition est fausse
}
```

```
if (condition1) {
    // bloc d'instructions à exécuter si la condition1 est vraie
} else if (condition2) {
    // bloc d'instructions à exécuter si la condition2 est vraie et la condition1 est fausse
} else {
    // bloc d'instructions à exécuter si la condition2 est fausse
}
```

Structures itératives

La boucle For

```
for (let i = 0; i < 5; i++) {  
  text += "Le nombre est " + i + "<br>";  
}
```

Array.forEach()

```
const numbers = [45, 4, 9, 16, 25];  
let txt = "";  
numbers.forEach(myFunction);  
function myFunction(value, index) {  
  txt += value;  
}
```

Of

```
const langages = ["Java", "Python", "C++"];  
let text = "";  
for (let x of langages) {  
  text += x;  
}
```

La boucle For in

```
const person = {fname:"Hassan",  
lname:"FILALI", age:25};  
let text = "";  
for (let x in person) {  
  text += person[x];  
}
```

La déclaration Break / La déclaration continue

Break

```
for (let i = 0; i < 10; i++)  
{  
    if (i === 3) { break; }  
    text += "The number is " + i + "<br>";  
}
```

Continue

```
for (let i = 0; i < 10; i++)  
{  
    if (i === 3) { continue; }  
    text += "The number is " + i + "<br>";  
}
```

Les fonctions

- Une fonction est définie comme un bloc de code organisé et réutilisable, créé pour effectuer une action unique
- Le rôle des fonctions est d'offrir une meilleure modularité au code et un haut degré de réutilisation ;
- Une fonction JavaScript est un ensemble d'instructions qui peut prendre des entrées de l'utilisateur, effectuer un traitement et renvoyer le résultat à l'utilisateur ;
- Une fonction JavaScript est exécutée au moment de son appel.

```
function nomFonction(paramètre1, paramètre2, paramètre3, ...)  
{  
    // Code de la fonction  
    return ... ;  
}
```

Les expressions lambdas

- Les fonctions fléchées (arrow functions) sont des fonctions qui ont une syntaxe compacte. Par conséquent, elles sont plus rapide à écrire que les fonctions traditionnelles ;
- Les fonctions fléchées sont limitées et ne peuvent pas être utilisées dans toutes les situations ;
- Principe: à la place du mot clé function, on utilise le signe (=>) plus une parenthèse carrée fermante (>) après la parenthèse fermante de la fonction :

```
const variable = () => {  
    return "ma_variable"  
}  
console.log(variable()) // "ma_variable"
```

```
const variable = (a,b) => {  
    return a*b;  
}  
console.log(variable(2,3)) // 6
```

Les expressions lambdas (exemples)

```
const langages = [  
  'Java',  
  'Python',  
  'PHP',  
  'Scala'  
];  
console.log(langages.map(L => L.length));
```

```
const langages = [  
  'Java',  
  'Python',  
  'PHP',  
  'Scala'  
];  
  
langages.forEach(L=>{  
  if (L.length>4){  
    console.log(L);  
  }  
})
```

Fonction de rappel (Callback)

- Les fonctions JavaScript sont exécutées dans l'ordre où elles sont appelées. Pas dans l'ordre où elles sont définies ;
- Une fonction peut appeler une fonction dans son code, directement ou en utilisant les fonctions de rappel ;
- Une fonction de rappel (callback en anglais) est une fonction passée en paramètre d'une autre fonction en tant qu'argument ;
- La fonction de rappel est invoquée à l'intérieur de la fonction externe pour exécuter des instructions précises

Exemples:

```
function affichage(s) {  
    console.log(s);  
}  
function calcul(num1, num2, myCallback) {  
    let somme = num1 + num2;  
    myCallback(somme);  
}  
calcul(1, 5, affichage);
```

Gestion des exceptions

Lorsqu'une erreur se produit, JavaScript arrête et crée un objet d'erreur (error) avec deux propriétés : nom et un message (varie selon le navigateur) → JavaScript lance une exception, on dit qu'une erreur est générée.

Pour gérer les exceptions en JavaScript on utilise les mots clés suivants :

- try : tester un bloc de code pour les erreurs ;
- catch : gérer l'erreur ;
- throw : créer des erreurs personnalisées ;
- Finally : permet l'exécution de code, après try et catch, quel que soit le résultat.

```
try {  
    //Bloc du code à exécuter  
}  
catch(err) {  
    //Bloc du code qui gère l'exception  
}
```

Gestion des exceptions

L'instruction throw

L'instruction throw permet de créer une erreur personnalisée.

L'exception à générer peut être de type String, Number, un Boolean ou un Object

Exemple :

```
if(a > 200)
    throw "Too big"; // exception de type string
else
    throw 500; // exception de type number
```

Les tableaux : déclaration

Un tableau JS est un objet qui hérite de l'objet global standard Array (héritant de Object).
L'objet Array est une liste d'éléments indexés dans lesquels on pourra ranger (écrire) et reprendre (lire) des données.

Déclaration d'un tableau vide :

```
let tableau = new Array; // déclaration explicite en instanciant l'objet Array  
let tableau = new Array(3); // 3 correspond à la taille du tableau  
let tableau = []; // déclaration de manière littérale avec []
```

Déclaration et initialisation

```
let tableau = new Array(5, 10, 15, 20);  
let tableau = ['A', 'B', 'C'];
```

Taille du tableau : La propriété length de l'objet Array retourne la taille du tableau

```
let tableau = ['A', 'B', 'C'];  
let nbElements = tableau.length;  
console.log(nbElements); // 3
```

Accéder aux éléments du tableau

Pour accéder à un élément du tableau, on utilise son indice : `nom_du_tableau[i] = "élément"`;

Rappel : Les indices du tableau commencent à 0

Accéder à un élément du tableau :

```
let tableau = ['A', 'B', 'C'];  
console.log(tableau[1]); // Retourne 'B'
```

Récupérer l'indice d'un élément : la méthode `indexOf()` :

```
let tableau = ['A', 'B', 'C'];  
console.log(tableau.indexOf('C')); // Retourne 2
```

`indexOf()` a un deuxième paramètre optionnel qui permet aussi de choisir l'indice à partir duquel la recherche débute (Par défaut, ce deuxième paramètre est à 0) :

```
let tableau = ['A', 'B', 'C', 'B'];  
console.log(tableau.indexOf('B')); // Retourne 1, l'indice du premier B trouvé  
console.log(tableau.indexOf('B', 2)); // Retourne 3, l'indice du premier B trouvé après l'indice 2
```

Parcourir un tableau

Pour accéder à un élément du tableau, on utilise son indice : `nom_du_tableau[i] = "élément";`

Rappel : Les indices du tableau commencent à 0

```
for (let i = 0; i < monTableau.length; i++)  
{  
    // monTableau[i] permet d'accéder à l'élément courant du tableau  
}  
//Ou bien  
for (const monElement of monTableau)  
{  
    // monElement permet d'accéder à l'élément courant du tableau  
}
```

```
for (let i = 0; i < monTableau.length; i++)  
{  
    // monTableau[i] permet d'accéder à l'élément courant du tableau  
}  
  
//Ou bien  
for (const monElement of monTableau)  
{  
    // monElement permet d'accéder à l'élément courant du tableau  
}
```

Ajouter un élément dans un tableau

La méthode push ajoute un élément à la fin du tableau

```
let tableau = ['A', 'B'];  
tableau.push('C');  
console.log(tableau); // Retourne ['A', 'B', 'C']
```

La méthode unshift ajoute un élément au début du tableau

```
let tableau = ['A', 'B'];  
tableau.unshift(22);  
console.log(tableau); // Retourne [22, 'A', 'B'];
```

Modifier un élément du tableau

Pour modifier la valeur d'un élément, on peut directement utiliser son indice

```
let tableau = ['B', 'B', 'C'];  
tableau[0] = 'A'; // Modifie la première case  
tableau[4] = 'E'; // ajoute un élément dans l'indice 4  
console.log(tableau); // Retourne ['A', 'B', 'C', empty, 'E']
```

Supprimer un élément du tableau

La méthode `pop()` supprime le dernier élément du tableau

```
let tableau = ['A', 'B', 'C'];  
tableau.pop();  
console.log(tableau); // Retourne ['A', 'B'];
```

La méthode `shift()` supprime le premier élément du tableau

```
let tableau = ['A', 'B', 'C'];  
tableau.shift();  
console.log(tableau); // Retourne ['B', 'C'];
```

La méthode `splice()` permet de supprimer plusieurs éléments

```
let tableau = ['A', 'B', 'C'];  
tableau.splice(1,1);  
console.log(tableau); // Retourne ['A', 'C'];
```

Trier un tableau

La méthode `sort()` retourne les éléments par ordre alphabétique (elle doivent être de la même nature)

```
let tableau = ['E', 'A', 'D'];  
tableau.sort();  
console.log(tableau); // Retourne ['A', 'D', 'E']
```

La méthode `reverse()` Inverse l'ordre des éléments (sans tri)

```
let tableau = ['A', 'B', 'C'];  
tableau.reverse();  
console.log(tableau); // Retourne ['C', 'B', 'A'];
```

La méthode `splice()` permet de supprimer plusieurs éléments

```
let tableau = ['A', 'B', 'C'];  
tableau.splice(1,1);  
console.log(tableau); // Retourne ['A', 'C'];
```

L'objet String (chaînes de caractères)

L'objet String de JavaScript est utilisé pour manipuler les chaînes de caractères. Il possède de nombreuses méthodes :

Méthode	Description
<code>length</code>	C'est un entier qui indique la taille de la chaîne de caractères.
<code>charAt()</code>	Méthode qui permet d'accéder à un caractère isolé d'une chaîne.
<code>substring(x,y)</code>	Méthode qui renvoie un string partiel situé entre la position x et la position y-1.
<code>toLowerCase()</code>	Transforme toutes les lettres en minuscules.
<code>toUpperCase()</code>	Transforme toutes les lettres en Majuscules.

- Chercher dans une Chaîne : `indexOf`, `startsWith`, `endsWith`, `split`, ...
- Transformer une chaîne en tableau : la méthode `Array.from()` permet de transformer une chaîne en un véritable tableau

L'objet Date

L'objet date de JavaScript est utilisé pour obtenir la date (année, mois et jour) ainsi que le temps (heure, minutes et secondes).

On peut utiliser 4 variantes du constructeur Date pour créer un objet date

Date()

Date (millisecondes)

Date (chaîne de date)

**Date (année, mois, jour,
heures, minutes, secondes,
millisecondes)**

Méthodes de l'objet Date JavaScript

Méthodes	Description
<code>getDate()</code>	Renvoie la valeur entière comprise entre 1 et 31 qui représente le jour de la date spécifiée sur la base de l'heure locale.
<code>getDay()</code>	Renvoie la valeur entière comprise entre 0 et 6 qui représente le jour de la semaine sur la base de l'heure locale.
<code>getFullYear()</code>	Renvoie la valeur entière qui représente l'année sur la base de l'heure locale.
<code>getHours()</code>	Renvoie la valeur entière comprise entre 0 et 23 qui représente les heures sur la base de l'heure locale.
<code>getMilliseconds()</code>	Renvoie la valeur entière comprise entre 0 et 999 qui représente les millisecondes sur la base de l'heure locale.
<code>getMinutes()</code>	Renvoie la valeur entière comprise entre 0 et 59 qui représente les minutes sur la base de l'heure locale.
<code>getMonth()</code>	Renvoie la valeur entière comprise entre 0 et 11 qui représente le mois sur la base de l'heure locale.
<code>getSeconds()</code>	Renvoie la valeur entière comprise entre 0 et 60 qui représente les secondes sur la base de l'heure locale.
<code>getUTCDate()</code>	Renvoie la valeur entière comprise entre 1 et 31 qui représente le jour de la date spécifiée sur la base de l'heure universelle.
<code>getUTCDay()</code>	Renvoie la valeur entière comprise entre 0 et 6 qui représente le jour de la semaine sur la base de l'heure universelle.
<code>getUTCFullYear()</code>	Renvoie la valeur entière qui représente l'année sur la base du temps universel.
<code>getUTCHours()</code>	Renvoie la valeur entière comprise entre 0 et 23 qui représente les heures sur la base du temps universel.
<code>getUTCMinutes()</code>	Renvoie la valeur entière comprise entre 0 et 59 qui représente les minutes sur la base du temps universel.
<code>getUTCMonth()</code>	Renvoie la valeur entière comprise entre 0 et 11 qui représente le mois sur la base du temps universel.
<code>getUTCSeconds()</code>	Renvoie la valeur entière comprise entre 0 et 60 qui représente les secondes sur la base du temps universel.
<code>setDate()</code>	Définit la valeur du jour pour la date spécifiée sur la base de l'heure locale.
<code>setDay()</code>	Définit le jour particulier de la semaine sur la base de l'heure locale.
<code>setFullYear()</code>	Définit la valeur de l'année pour la date spécifiée sur la base de l'heure locale.

L'objet Math

Méthodes	Description
abs()	Renvoie la valeur absolue du nombre donné.
acos(), asin(), atan()	Renvoie respectivement l'arc cosinus, l'arc sinus et l'arc tangente du nombre donné en radians.
ceil()	Renvoie une plus petite valeur entière, supérieure ou égale au nombre donné.
exp()	Renvoie la forme exponentielle du nombre donné.
floor()	Renvoie la plus grande valeur entière, inférieure ou égale au nombre donné.
log()	Renvoie le logarithme népérien d'un nombre.
max(), min()	Renvoie respectivement la valeur maximale et minimale des nombres donnés.
pow()	Renvoie la valeur de la base à la puissance de l'exposant.
random()	Renvoie un nombre aléatoire compris entre 0 (inclus) et 1 (exclusif).
round()	Renvoie la valeur entière la plus proche du nombre donné.
sin(), cos(), tan()	Renvoie respectivement le sinus, le cosinus et la tangente du nombre donné.
sqrt()	Il renvoie la racine carrée du nombre donné
trunc()	Il renvoie une partie entière du nombre donné.

L'objet Window

L'objet window de JavaScript est le parent de chaque objet qui compose la page web. Il possède plusieurs méthodes :

La méthode alert : bloque le programme tant que l'utilisateur n'aura pas cliqué sur "OK". Utile pour le déboguer les scripts.

```
alert(variable) ;  
alert("chaîne de caractères") ;  
alert(variable+ "chaîne de caractères");
```

La méthode confirm(texte) : permet d'avertir l'utilisateur en ouvrant une boîte de dialogue avec deux choix "OK" et "Annuler ". Le clic sur OK renvoie la valeur true

```
if (confirm('texte'))  
{ //action à faire pour la valeur true }  
else  
{ //action à faire pour la valeur false }
```

La méthode prompt (boîte d'invite) propose un champ comportant une entrée à compléter par l'utilisateur. Cette entrée possède aussi une valeur par défaut. En cliquant sur OK, la méthode renvoie la valeur tapée par l'utilisateur ou la réponse proposée par défaut. Si l'utilisateur clique sur Annuler ou Cancel, la valeur null est alors renvoyée

```
prompt("texte de la boîte d'invite" ,"valeur par défaut");
```

L'objet Window

Les méthodes **setTimeout()** et **clearTimeout()** permettent de déclencher une fonction après un laps de temps déterminé.

```
nom_du_compteur = setTimeout("fonction_appelée()", temps en milliseconde)
```

Exemple : lancer la fonction démarrer() après 5 secondes

```
setTimeout("demarrer()", 5000)
```

Pour arrêter le temporisateur avant l'expiration du délai fixé

```
clearTimeout(nom_du_compteur);
```

L'objet Window

La méthode **setInterval** () appelle une fonction ou évalue une expression à des intervalles spécifiés (en millisecondes).

La méthode `setInterval` () continue d'appeler la fonction jusqu'à ce que la méthode `clearInterval` () soit appelée ou que la fenêtre soit fermée

```
var x = setInterval(fonction, temps)
...
clearInterval(x)
```

Les expressions régulières (regex)

- Les expressions régulières (Regular Expressions - Regex) sont des patrons (exprimés sous forme de combinaisons de caractères) permettant d'effectuer des opérations de recherche et de remplacement sur un texte.
- En JavaScript, les expressions régulières sont des objets (RegExp Object) possédant des propriétés et des méthodes.

Syntaxe :

Pour créer une expression régulière en JavaScript, il faut entourer le patron (pattern) par les caractères (/)

```
let Expr = /eniad/;
```

Ou bien en utilisant le constructeur RegExp de JavaScript

```
let Expr = new RegExp("eniad");
```

Les expressions régulières (regex)

Les méthodes utilisées dans les expressions régulières sont listées dans le tableau ci-dessous:

Méthode	Description
<code>exec()</code>	Cherche une correspondance (match) d'un pattern dans une chaîne de caractères. Retourne un tableau ou null.
<code>test()</code>	Cherche une correspondance (match) d'un pattern dans une chaîne de caractères. Retourne true ou false.
<code>match()</code>	Retourne null ou un tableau contenant toutes les correspondances.
<code>matchAll()</code>	Retourne un itérateur contenant toutes les correspondances.
<code>search()</code>	Teste une correspondance dans une chaîne de caractères. Retourne -1 ou l'index de la correspondance.
<code>replace()</code>	Cherche une correspondance dans une chaîne et la remplace par une sous-chaîne.
<code>replaceAll()</code>	Recherche toutes les correspondances dans une chaîne et les remplace par une sous-chaînes.
<code>split()</code>	Décompose une chaîne en un tableau de sous-chaînes selon une expression régulière.

La POO avec JavaScript: Création d'objet avec syntaxe littérale

La syntaxe littérale consiste à créer un objet en définissant ses propriétés à l'intérieur d'une paire d'accolades : { ... } :

Syntaxe

```
const monObjet = {  
  propriete1: valeur1,  
  propriete2: valeur2,  
  // ... ,  
  methode1(/* ... */) { ... // ..},  
  methode2(/* ... */) { ... // ...}  
  // ...  
};
```

Exemple

```
const telephone = {  
  marque: 'SmartF22',  
  prix: 400,  
  stock: 200,  
  ref: 'SmartPhone2022',  
  VerifierStock: function() {  
    if (this.stock > 0) {return true;}  
    else { return false; }  
  }  
}  
//Test :  
console.log(telephone.marque); //SmartF22  
console.log(telephone.VerifierStock()); //True
```

La POO avec JavaScript: Création d'objet avec syntaxe littérale

La création des objets en utilisant un constructeur permet d'instancier cet objet dans des variables différentes

Syntaxe

```
function monObjet(param1, param2, ...){  
  this.propriete1: param1,  
  this.propriete2: param2,  
  // ... ,  
  this.methode1 = function ()  
  {... // ..},  
  // ...  
};
```

Exemple

```
function Telephone(n, p, s, r) {  
  this.nom = n;  
  this.prix = p;  
  this.stock = s;  
  this.ref = r;  
  this.VerifierStock = function() {  
    if (this.stock > 0) {return true;}  
    else {return false;}  
  }  
}  
//Test :  
var t1 = new Telephone("SmartF22", 400, 200, "pro1");  
var t2= new Telephone("t2", 200, 0, "Mi Max");  
console.log(t1.nom); //SmartF22  
console.log(t2.VerifierStock()); //False
```

Manipulation JSON

JSON (JavaScript Object Notation) est un format d'échange de données qui est facile à utiliser par les humains et les machines. Ce format est utilisé pour échanger les valeurs entre les applications (clients) et les serveurs. Sa forme complète est en notation d'objet JavaScript.

Caractéristiques de JSON :

- **Facile à utiliser** - l'API JSON offre une mise en œuvre avancée des différents types et structures de données, ce qui aide à simplifier les cas d'utilisation.
- **Performance et rapidité** - JSON consomme très peu de mémoire.
- **Outil gratuit** - la bibliothèque JSON est open source et gratuite.
- **Dépendance** - la bibliothèque JSON ne nécessite pas l'utilisation d'une autre bibliothèque pour le traitement.
- **Compatibilité :**
 - JSON est supporté par les navigateurs ;
 - JSON est pris en charge par tous les principaux framework JavaScript ;
 - JSON permet de transmettre et de sérialiser des données structurées à l'aide d'une connexion réseau ;
 - JSON est compatible avec des langages de programmation modernes.

Syntaxe JSON

Règles générales de la syntaxe JSON:

- Les données Json sont écrites entre accolades (braces) ;
- Les données sont représentées sous forme de paires de clé – valeur ;
- Les clés doivent être mises entre guillemets (double quotes) ;
- La clé et la valeur doivent être séparées par deux points (:)
- La virgule (,) est utilisée pour séparer les données ;
- Les crochets tiennent les tableaux (brackets) ;
- Les accolades retiennent les objets.

Les types JSON : Number, String (entre guillemets), Boolean, Null, Array, Object

```
{
  "nom" : "saidi",
  "prenom" : "ali",
  "age" :40,
  "interets" :null,
  "experience" : ["CSS","JS","HTML"],
  "adresse" : {"Rue" : "Sidi Maarouf","Ville": "Casablanca","codeP" : 10000 }
}
```

Manipulation des données JSON

Pour traiter et afficher les données JSON dans les pages web, on a souvent besoin de les convertir en objets Javascript et vice versa.

- Analyse syntaxique (parse) : Convertir une chaîne de caractères en un objet natif.
- Linéarisation (stringification) : Convertir un objet natif en chaîne de caractères.

En Javascript, les méthodes utilisées sont :

- JSON.parse permet de convertir JSON vers un objet javascript.
- JSON.stringify permet de convertir des objets javascript vers des données JSON.

Exemples:

```
//Création d'un string JSON
var jsonData = '{"nom":"Saidi", "prenom":"Ali"}';
document.write(typeof(jsonData)+'<br>'); //string
//Convertir JSON vers Javascript
var jsObject = JSON.parse(jsonData);
document.write(typeof(jsObject)+'<br>'); //object
document.write(jsObject+'<br>'); //[object object]
document.write(jsObject.nom +
""+jsObject.prenom+'<br>'); //Saidi Ali
```

```
//Création d'un objet Javascript
var jsObject = {nom:"Saidi", prenom:"ali"};
document.write(typeof(jsObject)+'<br>'); //Object
//Convertir javascript vers JSON
var jsonString = JSON.stringify(jsObject);
document.write(typeof(jsonString)+'<br>'); //string
document.write(jsonString); //{"nom":"Saidi","prenom":"ali"}
//remarquer la présence des "" dans les clés
```

Notion de l'arbre DOM

- DOM (Document Object Model) est une interface de programmation (API) normalisée par le W3C.
- Son rôle est d'accéder au contenu du navigateur web, et le modifier, en utilisant des scripts.
- Le DOM représente un document HTML sous forme d'un arbre d'objets (un paragraphe, une image, un style, etc).
- La modification du document HTML à l'aide du DOM consiste alors à ajouter ou supprimer des nœuds de l'arbre.
- DOM offre un ensemble de méthodes pour accéder aux éléments HTML.

Avec le modèle objet, JavaScript peut créer du contenu HTML dynamique en :

- Modifiant / Ajoutant / Supprimant les éléments HTML de la page ;
- Modifiant / Ajoutant / Supprimant les attributs des éléments HTML existants ;
- Modifiant les styles CSS de la page ;
- Réagissant aux événements HTML dans la page

Méthodes de recherche d'éléments html

Méthode	Description
<code>document.getElementById(id)</code>	Retourne un élément par la valeur de l'attribut ID
<code>document.getElementsByTagName(name)</code>	Retourne les éléments par nom de balise
<code>document.getElementsByClassName(name)</code>	Retourne les éléments par nom de classe CSS

```
<!DOCTYPE html>
<html>
<head></head>
<body>
  <h1 id="p1" class="c1">cours DOM JS</h1>
  <p class="c1">1er paragraphe</p>
  <p class="c2">2ème paragraphe</p>
</body>
</html>
```

```
let e1=document.getElementById("p1");
console.log(e1);//<h1 id="p1" class="c1">cours DOM
JS</h1>
let e2=document.getElementsByTagName("p");
console.log(e2);//[HTMLCollection(2) [p.c1, p]]
let e3=document.getElementsByClassName("c1");
console.log(e3);
//HTMLCollection(2) [h1#p1.c1, p.c1, p1: h1#p1.c1]
```

Méthodes d'ajout et suppression d'éléments

Méthode	Description
<code>document.createElement(element)</code>	Créer un élément HTML
<code>document.removeChild(element)</code>	Supprimer un élément HTML
<code>document.appendChild(element)</code>	Ajouter un élément HTML enfant
<code>document.replaceChild(new, old)</code>	Remplacer un élément HTML
<code>document.write(text)</code>	Écrire dans un document HTML
<code>document.getElementById(id).onclick = function(){code}</code>	Ajouter un événement de clic à l'élément sélectionné

Propriétés des éléments DOM

Méthode	Description
<code>element.innerHTML</code>	Permet de récupérer tout le contenu HTML d'un élément du DOM
<code>element.attribute</code>	Changer l'attribut d'un élément
<code>element.style.property</code>	Changer le style d'un élément
<code>element.textContent</code>	Renvoie tout le contenu textuel d'un élément du DOM, sans le balisage HTML
<code>element.classList</code>	Permet de récupérer la liste des classes d'un élément du DOM

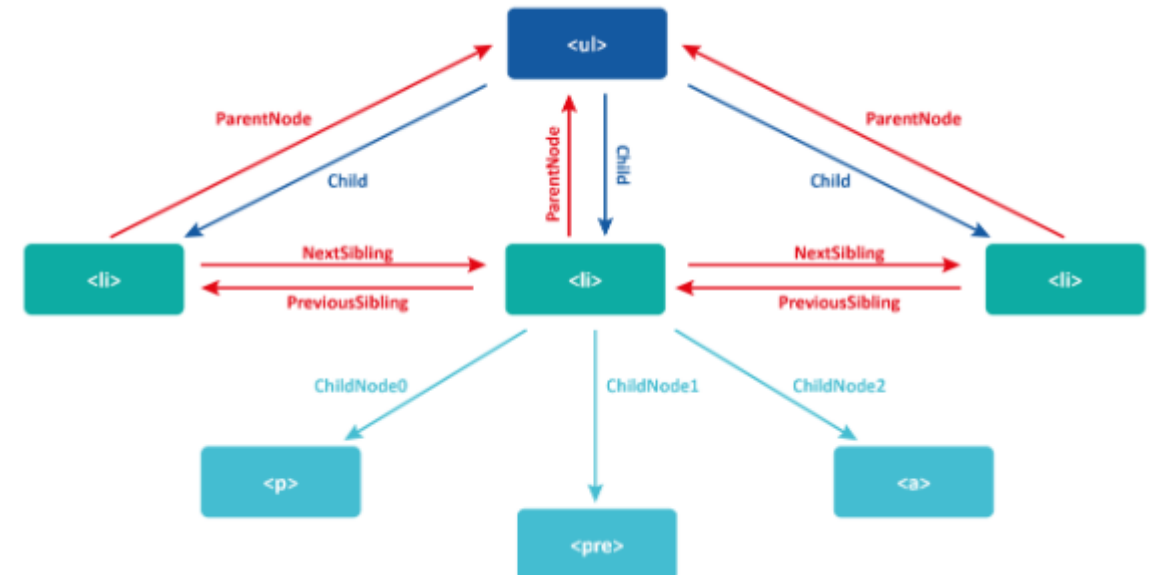
Relations entre les nœuds

Les éléments du DOM sont appelés des nœuds, qui sont en relation hiérarchique sous forme d'un arbre. Le nœud supérieur est appelé racine (ou nœud racine).

La relation entre les nœuds peut être qualifiée de relation :

- **Parent / Child**: des nœuds peuvent avoir des ascendants et des descendants
- **Nœuds ascendants** sont les nœuds qui sont parents d'un nœud (ou parents d'un nœud parent) ;
- **Nœuds descendants** sont les nœuds qui sont enfants d'un nœud (ou enfants d'un nœud enfant) ;
- Chaque nœud a exactement un parent, sauf la racine.
- Un nœud peut avoir plusieurs enfants.
- **Sibling** : correspondent aux frères d'un nœud, c'à-d les nœuds avec le même parent

```
<ul>
  <li>1er élément</li>
  <li>
    2ème élément
    <p>paragraphe</p>
    a>Lien</a>
  </li>
  <li>3ème élément</li>
</ul>
```



Navigation entre les nœuds de l'arborescence DOM

Navigation dans les nœuds enfants:

- **firstChild** : Retourne le premier enfant de l'élément.
- **firstElementChild** : Retourne le premier élément enfant du parent.
- **lastChild** : Retourne le dernier enfant de l'élément.
- **lastElementChild** : Retourne le dernier élément enfant du parent.
- **childNodes** : Retourne tous les enfants de l'élément sous forme d'une collection.
- **children** : Renvoie tous les enfants qui sont des éléments sous forme d'une collection.

Navigation dans les nœuds parents:

- **parentNode** : Renvoie le nœud parent de l'élément.
- **parentElement** : Renvoie le nœud de l'élément parent de l'élément.

Navigation dans les nœuds frères:

- **nextSibling** : Renvoie le nœud frère correspondant au prochain enfant du parent.
- **nextElementSibling** : Renvoie l'élément frère correspondant au prochain enfant de son parent.
- **previousSibling** : Renvoie le nœud frère qui est un enfant précédent de son parent.
- **previousElementSibling** : Renvoie l'élément frère qui est un enfant précédent de son parent

Manipulation DOM: Sélecteurs (simples, multiples...)

- La méthode **querySelector()** renvoie le premier élément qui correspond à un ou plusieurs sélecteurs CSS spécifiés.
- La méthode **querySelectorAll()** renvoie tous les éléments correspondants

Manipulation DOM: Créer un élément en JavaScript

La méthode `createElement` permet de créer de nouveaux éléments dans le document.

La variable `element` renvoie la référence de l'élément créé

```
let element1 = document.createElement('p');
console.log(element1); // <p></p>
let element2 = document.createElement('div');
console.log(element2); // <div></div>
//la méthode createElement convertit le nom de l'élément en minuscule
let element3 = document.createElement('DIV');
console.log(element3); // <div></div>
```

Manipulation DOM: Ajouter un élément en JavaScript

Pour ajouter un élément à l'arborescence du DOM (après l'avoir créée), il faut l'attacher à un élément parent.

La méthode **append()** insère un objet en tant que dernier enfant d'un élément parent.

```
let parent = document.getElementById("parent"); // sélectionner un élément parent
let enfant = document.createElement("p"); // Créer un élément enfant
enfant.innerHTML = "C'est un nouveau élément"; // Ajouter un texte à l'élément créé
parent.append(enfant); // Attacher l'enfant à l'élément parent
```

Manipulation DOM: Supprimer un élément en JavaScript

La méthode **removeChild()** supprime un élément de la structure du DOM. Le nœud à supprimer est passé en argument à la méthode. Une référence vers le nœud supprimé est retournée à la fin.

Exemple :

```
let parent = document.getElementById("parent"); // sélectionner un élément parent
let enfant = document.getElementById("eltSupp"); // Sélectionner un élément enfant
parent.removeChild(enfant);
```

Manipulation DOM: Modifier un élément en JavaScript

La méthode **replaceChild()** remplace un nœud par un autre nœud dans le DOM.
Une référence vers le nœud remplacé est retournée à la fin

```
parent.replaceChild(nouveauElement, ancienElement)
```

```
let parent = document.getElementById("parent"); // sélectionner un élément parent
let AncienElement = document.getElementById("id1"); // sélectionner l'ancien élément
let nouvElement = document.createElement("h2"); // Créer un nouveau élément de type <h2>
nouvElement.innerHTML = "C'est le nouveau élément."
parent.replaceChild(nouvElement, AncienElement);
```

Manipulation DOM: Mettre à jour le style

Les propriétés `.style` ou `.className` appliquées sur un élément permettent de changer les styles CSS.

Exemple:

```
<div>
  <div>
    <label>Nom : </label><br>
    <input type="text" class="style1" id="b1">
  </div>
</div>
<script>
  // Modifier le style de l'élément qui a la l'attribut class=style1
  document.getElementsByClassName("style1").style.borderColor = "red";
</script>
```

Manipulation DOM: Mise à jour d'un attribut avec setAttribute

La méthode **setAttribute()** est utilisée pour définir un attribut à l'élément spécifié. Si l'attribut existe déjà, sa valeur est mise à jour. Sinon, un nouvel attribut est ajouté avec le nom et la valeur spécifiés

```
<button type="button" id="Btn">Click</button>
<script>
  // sélectionner l'élément
  let btn = document.getElementById("Btn");
  // Ajouter les attributs
  btn.setAttribute("class", "style1");
  btn.setAttribute("disabled", "");
</script>
```

Manipulation DOM: Suppression d'attributs d'éléments

La méthode **removeAttribute()** est utilisée pour supprimer un attribut d'un élément spécifié.

```
<a href="https://www.ump.ma/" id="lien">UMP</a>
<script>
  // sélectionner l'élément
  let lien = document.getElementById("lien");
  // Supprimer la valeur de l'attribut href
  lien.removeAttribute("href");
</script>
```

Gérer les événements utilisateur

Qu'est-ce qu'un événement ?

Un événement est une action effectuée soit par l'utilisateur soit par le navigateur.

Il existe plusieurs types d'événements : événement de souris, un événement de clavier, un événement de document, etc

Javascript offre des mécanismes de réaction aux événements.

Les événements sont généralement traités par une fonction, qui s'exécute après que l'événement soit produit

Méthode `addEventListener()`

La méthode `addEventListener` appliquée sur un élément DOM, lui ajoute un gestionnaire pour un événement particulier. Cette fonction sera appelée à chaque fois que l'événement se produit sur l'élément.

Syntaxe : la méthode prend deux paramètres : le type de l'événement et la fonction qui gère l'événement
`element.addEventListener(événement, fonction de rappel, useCapture);`

```
let element = document.getElementById("btn");
element.addEventListener("click", message);
//Fonction qui gère l'événement
function message() {
    alert("Vous avez cliqué!")
}
```

Gérer les événements utilisateur: Les événements de la souris

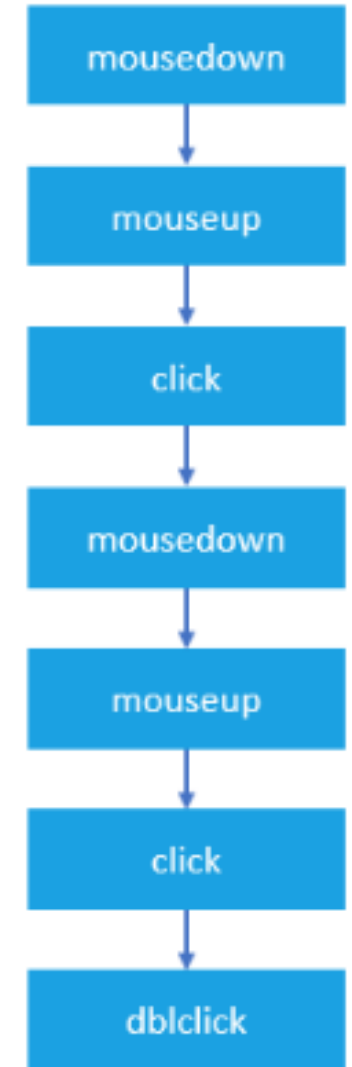
Les événements de la souris sont déclenchés quand elle interagit avec les éléments de la page.

Les événements DOM définissent neuf types d'événements de la souris : mousedown, mouseup, et click.

Quand la souris est enfoncée, trois événements se déclenchent dans l'ordre suivant :

- **Mousedown** : se déclenche lorsqu'on appuie sur le bouton gauche de la souris.
- **Mouseup** : se déclenche lorsqu'on relâche le bouton de la souris.
- **Click** : se déclenche lorsqu'un événement mousedown et un mouseup sont détectés sur l'élément

L'événement **dblclick** a quatre événements déclenchés dans l'ordre suivant :



Gérer les événements utilisateur: Les événements de la souris

- **mousemove**

L'événement `mousemove` se déclenche à plusieurs reprises lorsqu'on déplace le curseur de la souris autour d'un élément.

Pour ne pas ralentir la page, le gestionnaire de l'événement `mousemove` n'est enregistré qu'en cas de besoin, et doit être supprimé dès qu'il n'est plus utilisé.

Exemple :

- **mouseover / mouseout**

L'événement `mouseover` se déclenche lorsque le curseur de la souris se déplace à l'extérieur d'un élément.

L'événement `mouseout` se déclenche lorsque le curseur de la souris survole un élément, puis se déplace vers autre élément.

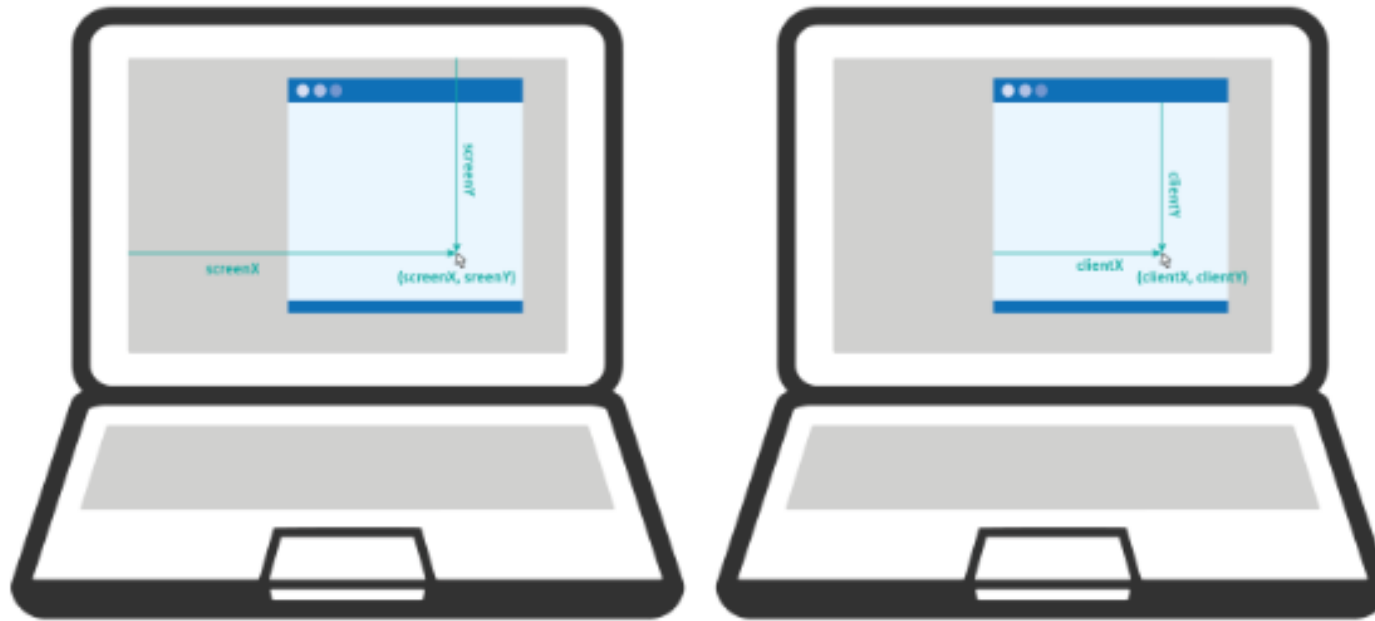
- **mouseenter / mouseleave**

L'événement `mouseenter` se déclenche lorsque le curseur de la souris se déplace de l'extérieur à l'intérieur d'un élément.

L'événement `mouseleave` se déclenche lorsque le curseur de la souris se déplace de l'intérieur à l'extérieur d'un élément.

Gérer les événements utilisateur: Obtenir les coordonnées de l'écran

- Les propriétés **screenX** et **screenY** de l'événement passées au gestionnaire d'événements de la souris renvoient les coordonnées (dans l'écran) correspondant à l'emplacement de la souris par rapport à l'ensemble de l'écran.
- Les propriétés **clientX** et **clientY** fournissent les coordonnées horizontales et verticales dans la zone cliente de l'application où l'événement de la souris s'est produit



Gérer les événements utilisateur: Les événements du clavier

Les propriétés de l'événement clavier:

L'événement clavier possède deux propriétés importantes :

- **key** : renvoie le caractère qui a été enfoncé.
- **Code** : renvoie le code de touche physique.

Par exemple, le clic sur la touche z du clavier : `event.key` retourne « z » et les « `event.code` » retourne « KeyZ ».